

Weekly Report 7
ML Code Crafters
Project 7: Offline Track association problem [UAV videos]

Name
Vidhan Nahar
Shubham Mehta
Jash Agicha
Yashvi Jakharia

Work Done This Week

- We introduced automated Parameter Tuning. This week, we focused on improving performance of our model and algorithm to identify most effective parameters and fine tune the model and pipeline deeply. We loop the procedure so instead of manually adjusting weight and trying to adjust the weight we automate the process by repeatedly changing weights and measuring accuracy.
- We explored with varying the types of parameters like motion score through kalman based prediction, IOU score from bounding boxes and cosine similarities. These different weights were considered to loop

This is the code for that implementation:

Python

```
# Automatic parameter tuning on the current sequence using the  
evaluation metrics  
  
def run_pipeline_and_evaluate(dist_threshold, time_gap_threshold,  
merge_distance_threshold):
```

```
"""Build tracklets and merged tracks with given parameters and
evaluate them.
```

```
Returns (eval_stats, tracklets_initial, tracklets_merged).
"""
```

```
if ann_filename is None:
    return None, None, None
```

```
# Rebuild tracklets from scratch for this setting
```

```
t_init = build_tracklets(df_ann, dist_threshold=dist_threshold)
```

```
t_merged = offline_merge_tracklets(
    t_init,
    time_gap_threshold=time_gap_threshold,
    distance_threshold=merge_distance_threshold,
)
```

```
eval_stats = evaluate_tracks_against_gt(t_merged, df_ann,
iou_threshold=0.5)
```

```
return eval_stats, t_init, t_merged
```

```
# Simple grid search over a small set of values
```

```
search_dist_thresholds = [40.0, 60.0, 80.0]
```

```
search_time_gaps = [3, 5, 8]
```

```
search_merge_dists = [40.0, 60.0, 80.0]
```

```
best_score = -1.0
```

```
best_cfg = None
```

```
best_eval = None
```

```
if ann_filename is not None:
```

```
    for d in search_dist_thresholds:
```

```
        for tg in search_time_gaps:
```

```
            for md in search_merge_dists:
```

```
                eval_stats, _, _ = run_pipeline_and_evaluate(d, tg,
md)
```

```
                    if eval_stats is None:
```

```
                        continue
```

```

        # Define a simple scalar score: high coverage, low
        fragmentation and ID switches
        score = (
            0.5 * eval_stats["coverage_gt"]
            + 0.5 * eval_stats["coverage_pred"]
            - 0.2 * max(0.0, eval_stats["avg_gt_fragments"]
- 1.0)
            - 0.2 * max(0.0,
eval_stats["avg_pred_id_switches"] - 1.0)
        )
        if score > best_score:
            best_score = score
            best_cfg = (d, tg, md)
            best_eval = eval_stats

    if best_cfg is not None:
        d, tg, md = best_cfg
        print("Best parameters on this sequence:")
        print(f"  dist_threshold (tracklets): {d}")
        print(f"  time_gap_threshold (merge): {tg}")
        print(f"  merge_distance_threshold: {md}")
        print(f"  score: {best_score:.3f}")
        print("  Eval summary for best params:")
        print(f"    coverage_gt:
{100*best_eval['coverage_gt']:.1f}%")
        print(f"    coverage_pred:
{100*best_eval['coverage_pred']:.1f}%")
        print(f"    avg_gt_fragments:
{best_eval['avg_gt_fragments']:.2f} (1.0 ideal)")
        print(f"    avg_id_switches:
{best_eval['avg_pred_id_switches']:.2f} (1.0 ideal)")
    else:
        print("Could not evaluate any configuration (no tracks or
annotations).")
    else:
        print("No annotation/sequence selected; cannot run parameter
search.")

```

Our Learnings

- Importance of Balanced and accurate weights: so increase in performance of model and pipeline when parameters are set effectively.
- Learned about the long-gap matching and accuracy trade off
- Manual Tuning can be ineffective

Challenges Faced

- How parameter tuning took such a long time because of how large the search space was making it computationally expensive
- Implementing proper evaluation metrics.
- Testing with more sequences.

Planning for Next Week

- Maybe a little change in our approach and add some kind of classifier for testing purposes.
- Integrate reliable evaluation metrics.
- Reduce computational time from the current fine tuner.